

Cryptography

Mark Stamp

Symmetric Key Crypto

- ❑ Stream cipher — generalize one-time pad
 - Except that key is relatively short
 - Key is stretched into a long **keystream**
 - Keystream is used just like a one-time pad
- ❑ Block cipher — generalized codebook
 - Block cipher key determines a codebook
 - Each key yields a different codebook
 - Employs both “confusion” and “diffusion”

Stream Ciphers



Stream Ciphers

- ❑ Once upon a time, not so very long ago... stream ciphers were the king of crypto
- ❑ Today, not as popular as block ciphers
- ❑ We'll discuss two stream ciphers:
- ❑ A5/1
 - Based on shift registers
 - Used in GSM mobile phone system
- ❑ RC4
 - Based on a changing lookup table
 - Used many places

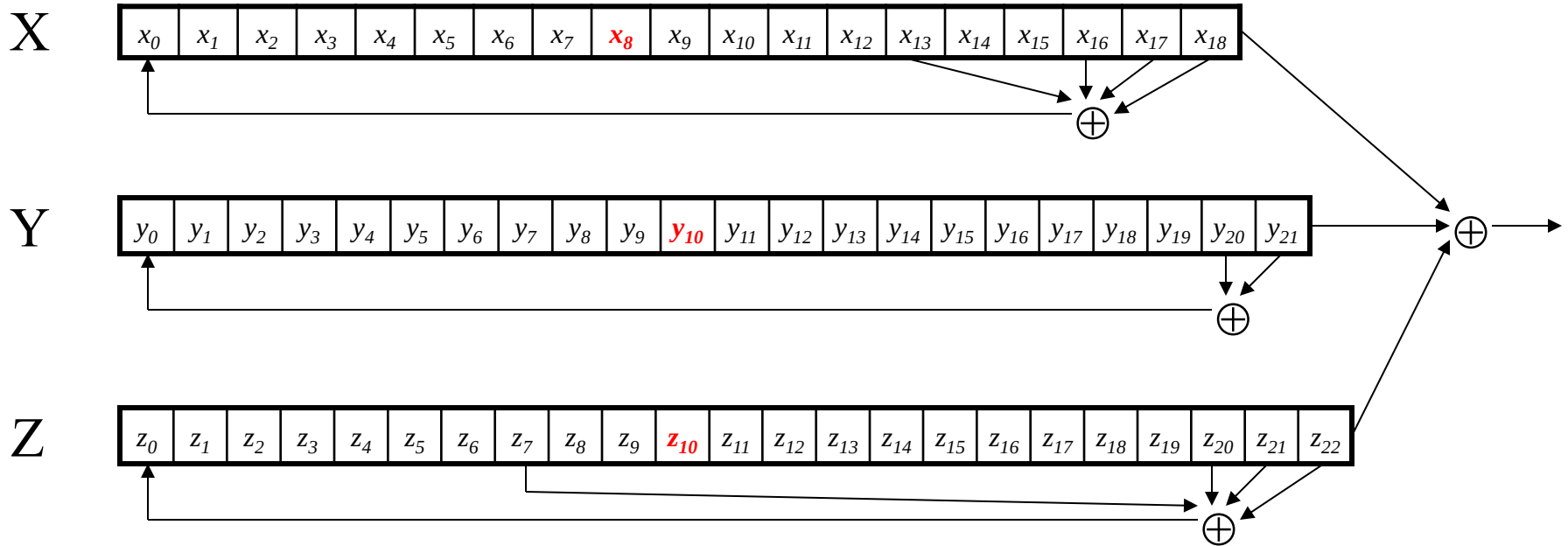
A5/1: Shift Registers

- *A5/1 uses 3 shift registers*
 - X: 19 bits ($x_0, x_1, x_2, \dots, x_{18}$)
 - Y: 22 bits ($y_0, y_1, y_2, \dots, y_{21}$)
 - Z: 23 bits ($z_0, z_1, z_2, \dots, z_{22}$)

A5/1: Keystream

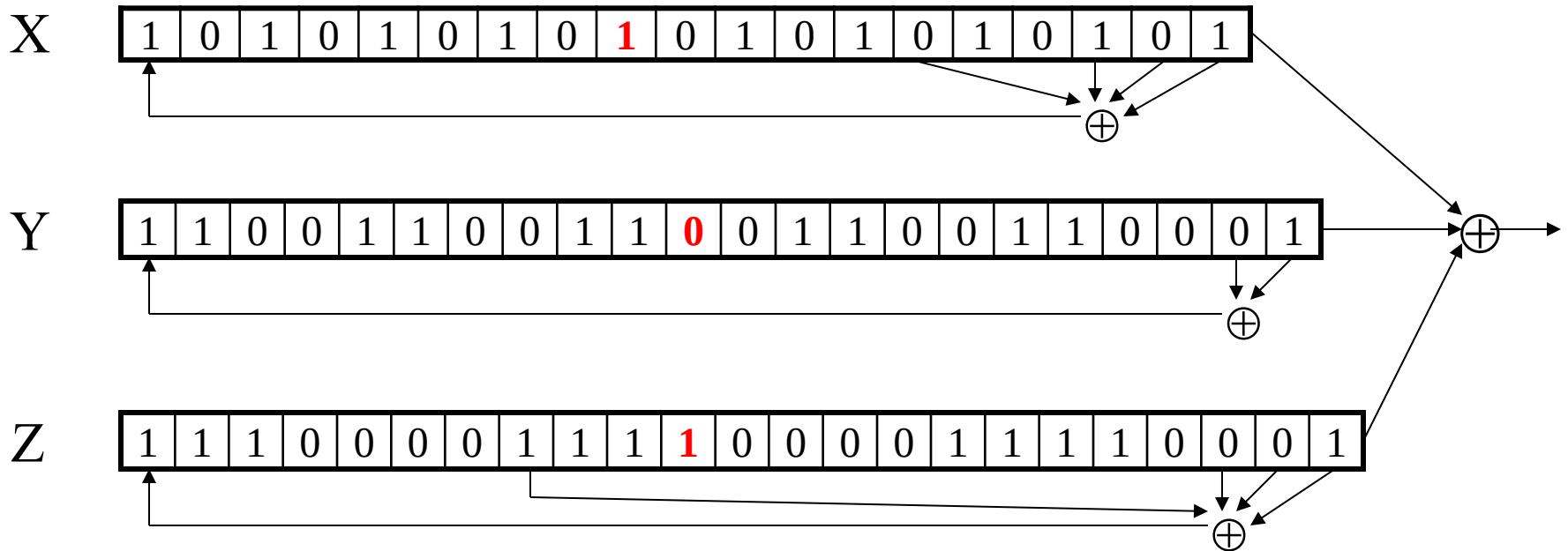
- ❑ At each iteration: $m = \text{maj}(x_8, y_{10}, z_{10})$
 - Examples: $\text{maj}(0,1,0) = 0$ and $\text{maj}(1,1,0) = 1$
- ❑ If $x_8 = m$ then **X steps**
 - $t = x_{13} \oplus x_{16} \oplus x_{17} \oplus x_{18}$
 - $x_i = x_{i-1}$ for $i = 18, 17, \dots, 1$ and $x_0 = t$
- ❑ If $y_{10} = m$ then **Y steps**
 - $t = y_{20} \oplus y_{21}$
 - $y_i = y_{i-1}$ for $i = 21, 20, \dots, 1$ and $y_0 = t$
- ❑ If $z_{10} = m$ then **Z steps**
 - $t = z_7 \oplus z_{20} \oplus z_{21} \oplus z_{22}$
 - $z_i = z_{i-1}$ for $i = 22, 21, \dots, 1$ and $z_0 = t$
- ❑ Keystream **bit** is $x_{18} \oplus y_{21} \oplus z_{22}$

A5/1



- ❑ Each variable here is a single bit
- ❑ Key is used as **initial fill** of registers
- ❑ Each register steps (or not) based on $\text{maj}(x_8, y_{10}, z_{10})$
- ❑ Keystream bit is XOR of rightmost bits of registers

A5/1



- ❑ In this example, $m = \text{maj}(x_8, y_{10}, z_{10}) = \text{maj}(\mathbf{1}, \mathbf{0}, \mathbf{1}) = \mathbf{1}$
- ❑ Register X steps, Y does not step, and Z steps
- ❑ Keystream bit is XOR of right bits of registers
- ❑ Here, keystream bit will be $0 \oplus 1 \oplus 0 = 1$

Shift Register Crypto

- ❑ Shift register crypto efficient in hardware
- ❑ Often, slow if implemented in software
- ❑ In the past, very, very popular
- ❑ Today, more is done in software due to fast processors
- ❑ Shift register crypto still used some
 - Especially in resource-constrained devices

RC4

- ❑ A self-modifying lookup table
- ❑ Table always contains a permutation of the byte values $0, 1, \dots, 255$
- ❑ Initialize the permutation using key
- ❑ At each step, RC4 does the following
 - Swaps elements in current lookup table
 - Selects a keystream byte from table
- ❑ Each step of RC4 produces a **byte**
 - Efficient in software
- ❑ Each step of A5/1 produces only a bit
 - Efficient in hardware

RC4 Initialization

- `S[]` is permutation of `0,1,...,255`
- `key[]` contains `N` bytes of key

```
for i = 0 to 255
    S[i] = i
    K[i] = key[i mod N]
next i
j = 0
for i = 0 to 255
    j = (j + S[i] + K[i]) mod 256
    swap(S[i], S[j])
next i
i = j = 0
```

RC4 Keystream

- At each step, swap elements in table and select keystream byte

```
i = (i + 1) mod 256
```

```
j = (j + S[i]) mod 256
```

```
swap(S[i], S[j])
```

```
t = (S[i] + S[j]) mod 256
```

```
keystreamByte = S[t]
```

- Use keystream bytes like a one-time pad
- **Note:** first 256 bytes should be discarded
 - Otherwise, related key attack exists

RC4 Simplified Example

- ❑ Instead of the full 256 bytes, we will use 8×3 -bits. That is, the state vector S is 8×3 -bits. We will operate on 3-bits of plaintext at a time since S can take the values 0 to 7, which can be represented as 3 bits.
- ❑ Assume we use a 4×3 -bit key of $K = [1\ 2\ 3\ 6]$. And a plaintext $P = [1\ 2\ 2\ 2]$
- ❑ Initialize the state vector S and temporary vector Key . S is initialized so the $S[i] = i$, and Key is initialized so it is the key K (repeated as necessary).
- ❑ $S = [0\ 1\ 2\ 3\ 4\ 5\ 6\ 7]$ $Key = [1\ 2\ 3\ 6\ 1\ 2\ 3\ 6]$
- ❑ $j = 0;$
- ❑ for $i = 0$ to 7 do
 - $j = (j + S[i] + Key[i]) \bmod 8$
 - $Swap(S[i], S[j]);$
 - $J = 1, 3, 0, 6, 3, 2, 5, 2$
 - $I = 0, 1, 2, 3, 4, 5, 6$
- ❑ Hence, our initial permutation of $S = [2\ 3\ 7\ 4\ 0\ 1\ 6\ 5];$

RC4 Simplified Example

- $S = [0\ 1\ 2\ 3\ 4\ 5\ 6\ 7]$ Key = $[1\ 2\ 3\ 6\ 1\ 2\ 3\ 6]$
- $j = 0;$
- for $i = 0$ to 7 do
 - $j = (j + S[i] + \text{Key}[i]) \bmod 8$
 - Swap($S[i], S[j]$);
 - S at the end of $i = 0$ iteration: $[1\ 0\ 2\ 3\ 4\ 5\ 6\ 7]$, $j = 1$
 - S at the end of $i = 1$ iteration: $[1\ 3\ 2\ 0\ 4\ 5\ 6\ 7]$, $j = 3$
 - S at the end of $i = 2$ iteration: $[2\ 3\ 1\ 0\ 4\ 5\ 6\ 7]$, $j = 0$
 - S at the end of $i = 3$ iteration: $[2\ 3\ 1\ 6\ 4\ 5\ 0\ 7]$, $j = 6$
 - S at the end of $i = 4$ iteration: $[2\ 3\ 1\ 4\ 6\ 5\ 0\ 7]$, $j = 3$
 - S at the end of $i = 5$ iteration: $[2\ 3\ 5\ 4\ 6\ 1\ 0\ 7]$, $j = 2$
 - S at the end of $i = 6$ iteration: $[2\ 3\ 5\ 4\ 6\ 0\ 1\ 7]$, $j = 5$
 - S at the end of $i = 7$ iteration: $[2\ 3\ 7\ 4\ 6\ 0\ 1\ 5]$, $j = 2$
 - Hence, our initial permutation of $S = [2\ 3\ 7\ 4\ 6\ 0\ 1\ 5];$

RC4 Simplified Example

- Now we generate 3-bits at a time, k , that we XOR with each 3-bits of plaintext to produce the ciphertext. The 3-bits k is generated by: $i, j = 0$; Consider that the initial permutation for S is $S = [2\ 3\ 7\ 4\ 0\ 1\ 6\ 5]$ when go through the following steps.
- $i = (i + 1) \bmod 8$;
- $j = (j + S[i]) \bmod 8$;
- Swap ($S[i], S[j]$);
- $t = (S[i] + S[j]) \bmod 8$;

The first iteration:

$S = [2\ 3\ 7\ 4\ 0\ 1\ 6\ 5]$
 $i = (0 + 1) \bmod 8 = 1$
 $j = (0 + S[1]) \bmod 8 = 3$
 Swap($S[1], S[3]$)
 $S = [2\ 4\ 7\ 3\ 0\ 1\ 6\ 5]$
 $t = (S[1] + S[3]) \bmod 8 = 7$
 $k = S[7] = 5$

Remember, $P = [1\ 2\ 2\ 2]$

So our first 3-bits of ciphertext is obtained by: $k \text{ XOR } P$
 $5 \text{ XOR } 1 = 101 \text{ XOR } 001 = 100 = 4$

The second iteration:

$S = [2\ 4\ 7\ 3\ 0\ 1\ 6\ 5]$
 $i = (1 + 1) \bmod 8 = 2$
 $j = (2 + S[2]) \bmod 8 = 1$
 Swap($S[2], S[1]$)
 $S = [2\ 7\ 4\ 3\ 0\ 1\ 6\ 5]$

$t = (S[2] + S[1]) \bmod 8 = 3$
 $k = S[3] = 3$

Second 3-bits of ciphertext are:
 $3 \text{ XOR } 2 = 011 \text{ XOR } 010 = 001 = 1$

The third iteration:

$S = [2\ 7\ 4\ 3\ 0\ 1\ 6\ 5]$
 $i = (2 + 1) \bmod 8 = 3$
 $j = (1 + S[3]) \bmod 8 = 4$
 Swap($S[3], S[4]$)
 $S = [2\ 7\ 4\ 0\ 3\ 1\ 6\ 5]$
 $t = (S[3] + S[4]) \bmod 8 = 3$
 $k = S[3] = 0$

Third 3-bits of ciphertext are:
 $0 \text{ XOR } 2 = 000 \text{ XOR } 010 = 010 = 2$

RC4 Simplified Example

- Now we generate 3-bits at a time, k , that we XOR with each 3-bits of plaintext to produce the ciphertext. The 3-bits k is generated by: $i, j = 0$;
- $i = (i + 1) \bmod 8$;
- $j = (j + S[i]) \bmod 8$;
- Swap ($S[i], S[j]$);
- $t = (S[i] + S[j]) \bmod 8$;
- $k = S[t]$;

The third iteration:

$S = [2\ 7\ 4\ 3\ 0\ 1\ 6\ 5]$

$i = (2 + 1) \bmod 8 = 3$

$j = (1 + S[3]) \bmod 8 = 4$

Swap($S[3], S[4]$)

$S = [2\ 7\ 4\ 0\ 3\ 1\ 6\ 5]$

$t = (S[3] + S[4]) \bmod 8 = 3$

$k = S[3] = 0$

Third 3-bits of ciphertext are:

$0 \text{ XOR } 2 = 000 \text{ XOR } 010 = 010 = 2$

The final iteration:

$S = [2\ 7\ 4\ 0\ 3\ 1\ 6\ 5]$

$i = (1 + 3) \bmod 8 = 4$

$j = (4 + S[4]) \bmod 8 = 7$

Swap($S[4], S[7]$)

$S = [2\ 7\ 4\ 0\ 5\ 1\ 6\ 3]$

$t = (S[4] + S[7]) \bmod 8 = 0$

$k = S[0] = 2$

Last 3-bits of ciphertext are:

$2 \text{ XOR } 2 = 010 \text{ XOR } 010 = 000 = 0$

So to encrypt the plaintext stream $P = [1\ 2\ 2\ 2]$ with key $K = [1\ 2\ 3\ 6]$ using our simplified RC4 stream cipher we get $C = [4\ 1\ 2\ 0]$.

(or in binary: $P = 001010010010$, $K = 001010011110$ and $C = 100001010000$)

Stream Ciphers

- ❑ Stream ciphers were popular in the past
 - Efficient in hardware
 - Speed was needed to keep up with voice, etc.
 - Today, processors are fast, so software-based crypto is usually more than fast enough
- ❑ Future of stream ciphers?
 - Shamir declared “the death of stream ciphers”
 - May be greatly exaggerated...

Block Ciphers



(Iterated) Block Cipher

- ❑ Plaintext and ciphertext consist of fixed-sized blocks
- ❑ Ciphertext obtained from plaintext by iterating a **round function**
- ❑ Input to round function consists of **key** and **output** of previous round
- ❑ Usually implemented in software

Feistel Cipher: Encryption

- ❑ **Feistel cipher** is a type of block cipher
 - **Not** a specific block cipher
- ❑ Split plaintext block into left and right halves: $P = (L_0, R_0)$
- ❑ For each round $i = 1, 2, \dots, n$, compute

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

where F is **round function** and K_i is **subkey**

- ❑ Ciphertext: $C = (L_n, R_n)$

Feistel Cipher: Decryption

- ❑ Start with ciphertext $C = (L_n, R_n)$
- ❑ For each round $i = n, n-1, \dots, 1$, compute
$$R_{i-1} = L_i$$
$$L_{i-1} = R_i \oplus F(R_{i-1}, K_i)$$
where F is round function and K_i is subkey
- ❑ Plaintext: $P = (L_0, R_0)$
- ❑ Decryption works for any function F
 - But only secure for certain functions F

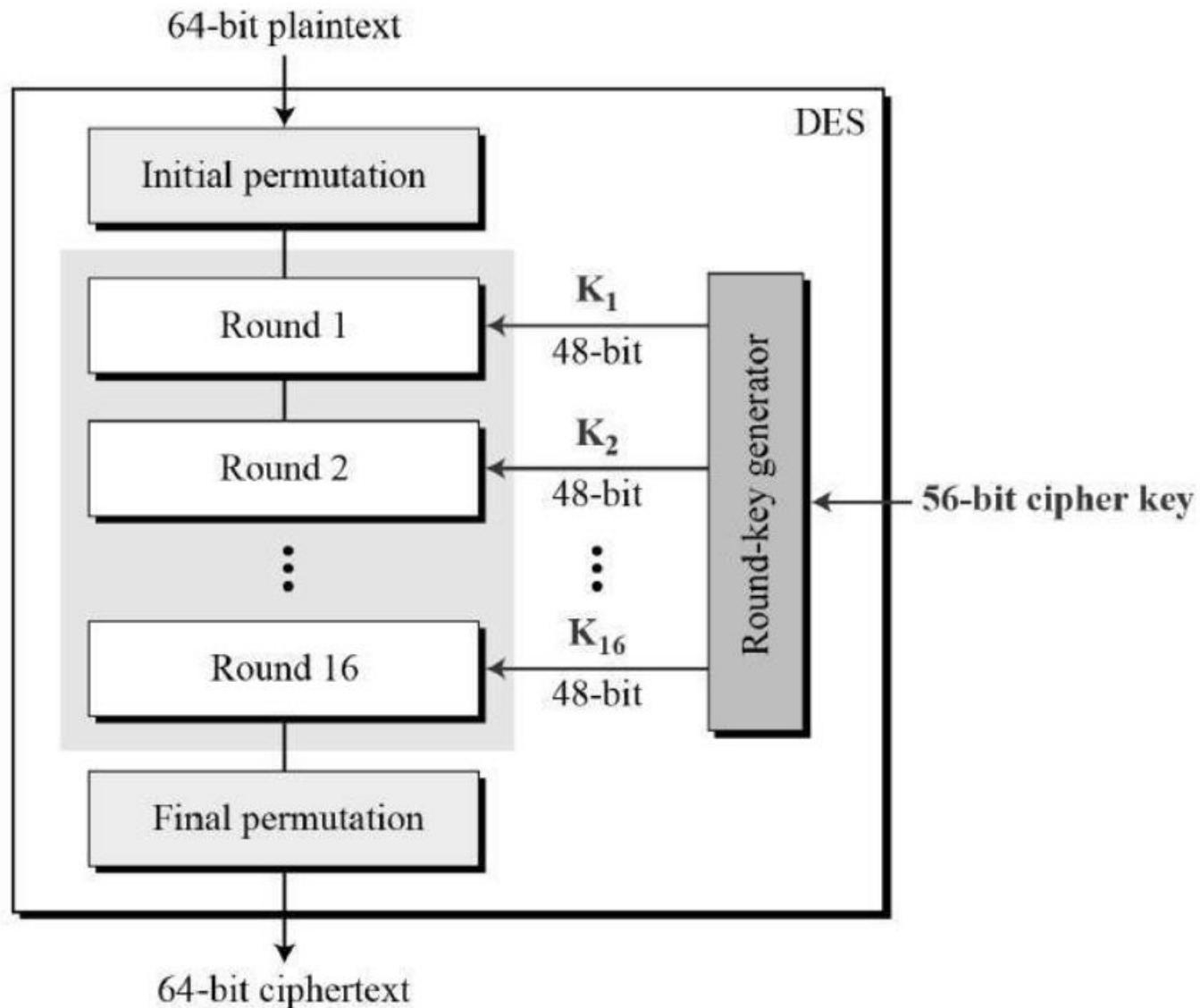
Data Encryption Standard

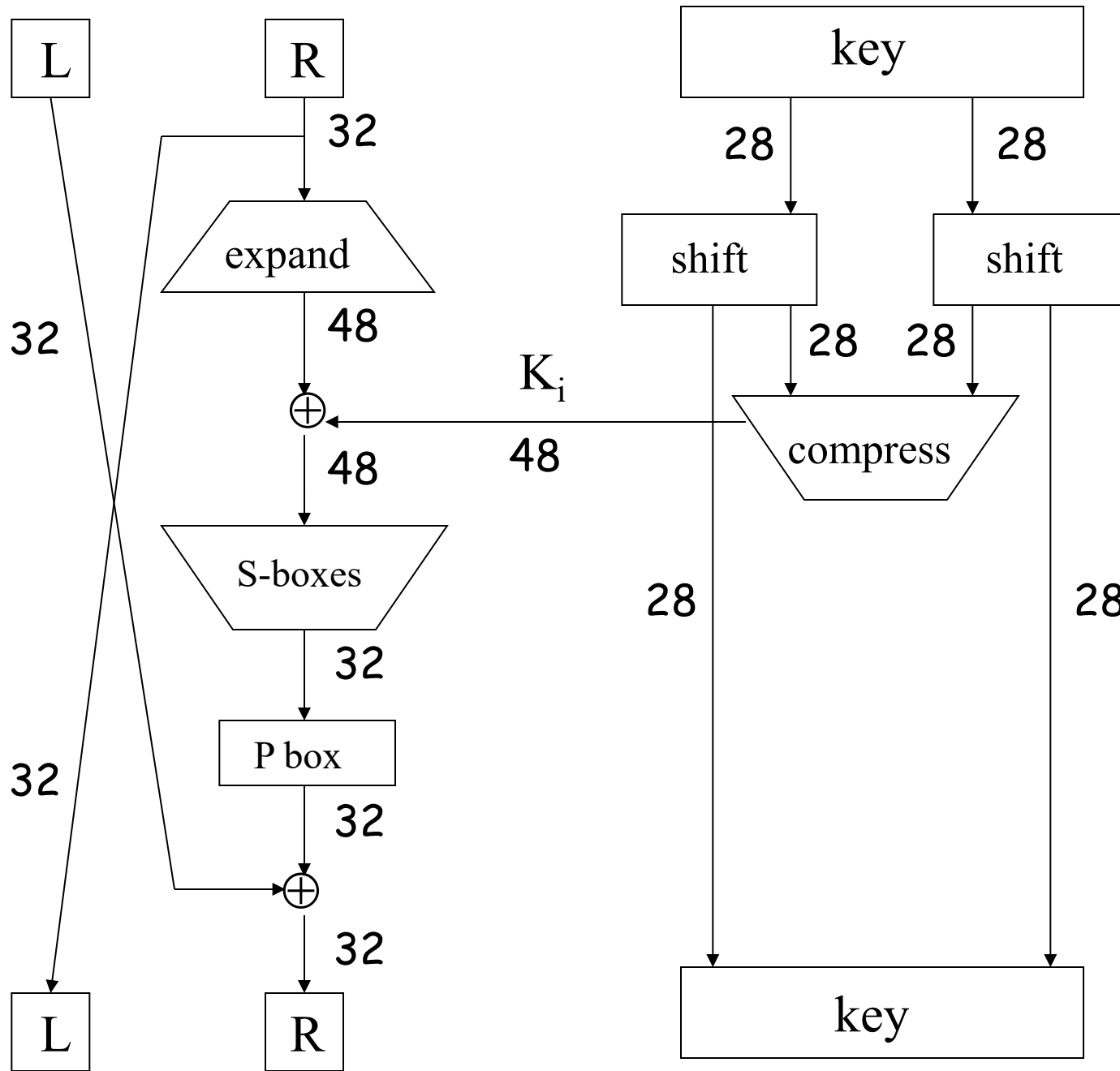
- ❑ **DES** developed in 1970's
- ❑ Based on IBM's Lucifer cipher
- ❑ DES was U.S. government standard
- ❑ Development of DES was controversial
 - NSA secretly involved
 - Design process was secret
 - Key length reduced from 128 to 56 bits
 - Subtle changes to Lucifer algorithm

DES Numerology

- ❑ DES is a Feistel cipher with...
 - 64 bit block length
 - 56 bit key length
 - 16 rounds
 - 48 bits of key used each round (subkey)
- ❑ Round function is simple (for block cipher)
- ❑ Security depends heavily on "S-boxes"
 - Each S-box maps 6 bits to 4 bits

General Structure of DES

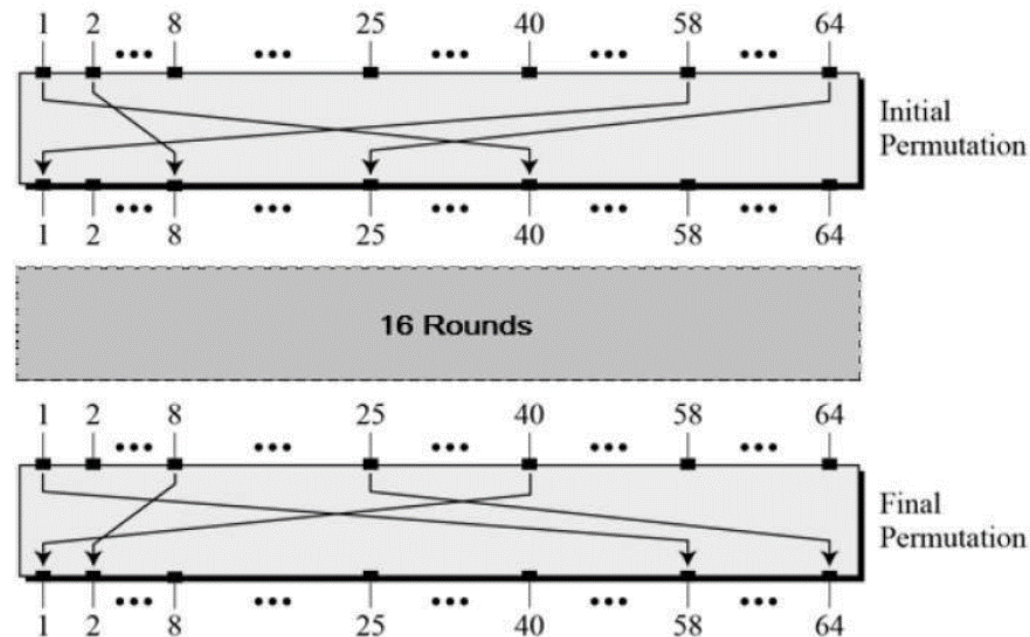




One
Round
of
DES

Initial and Final Permutation

- ❑ The initial and final permutations are straight Permutation boxes (P-boxes) that are inverses of each other.
- ❑ They have no cryptography significance in DES.



Initial Permutation

- ❑ This table specifies the input permutation on a 64-bit block.
- ❑ The meaning is as follows:
 - the first bit of the output is taken from the 58th bit of the input; the second bit from the 50th bit, and so on, with the last bit of the output taken from the 7th bit of the input.
- ❑ This information is presented as a table for ease of presentation, it is a vector, not a matrix.

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

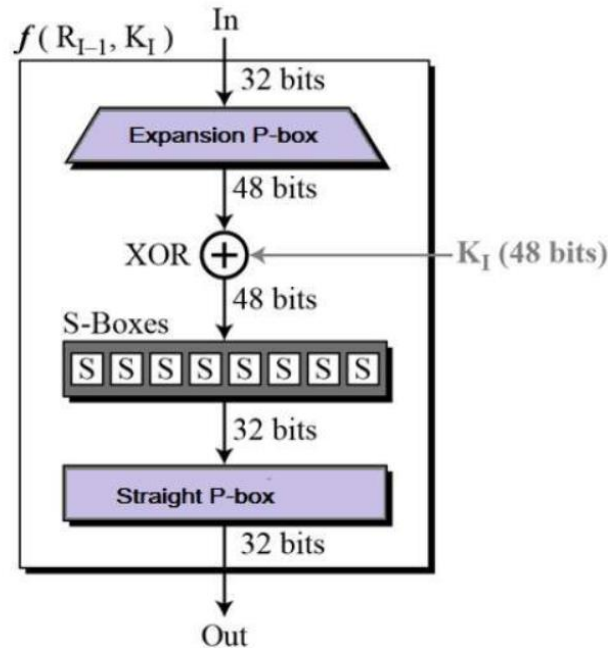
Final Permutation

- The final permutation is the inverse of the initial permutation; the table is interpreted similarly.
 - That is, the output of the Final Permutation has bit 40 of the preoutput block as its first bit, bit 8 as its second bit, and so on, until bit 25 of the preoutput block is the last bit of the output.

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

Round Function

- The DES function applies a 48-bit key to the rightmost 32 bits to produce a 32-bit output.

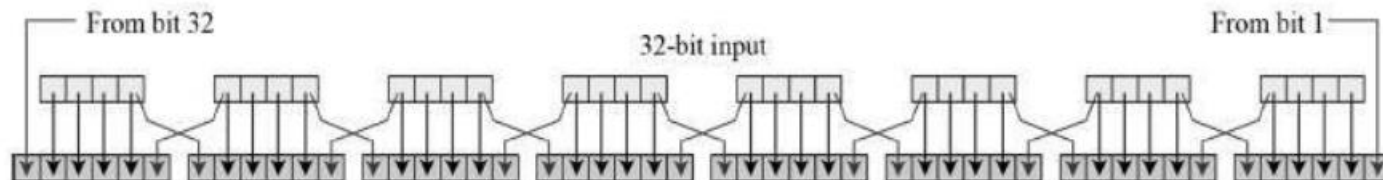


Round Function

- ❑ Expansion Permutation Box
- ❑ XOR (Whitener)
- ❑ Substitution Boxes
- ❑ Straight Permutation

DES Expansion Permutation

32	01	02	03	04	05
04	05	06	07	08	09
08	09	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	31	31	32	01



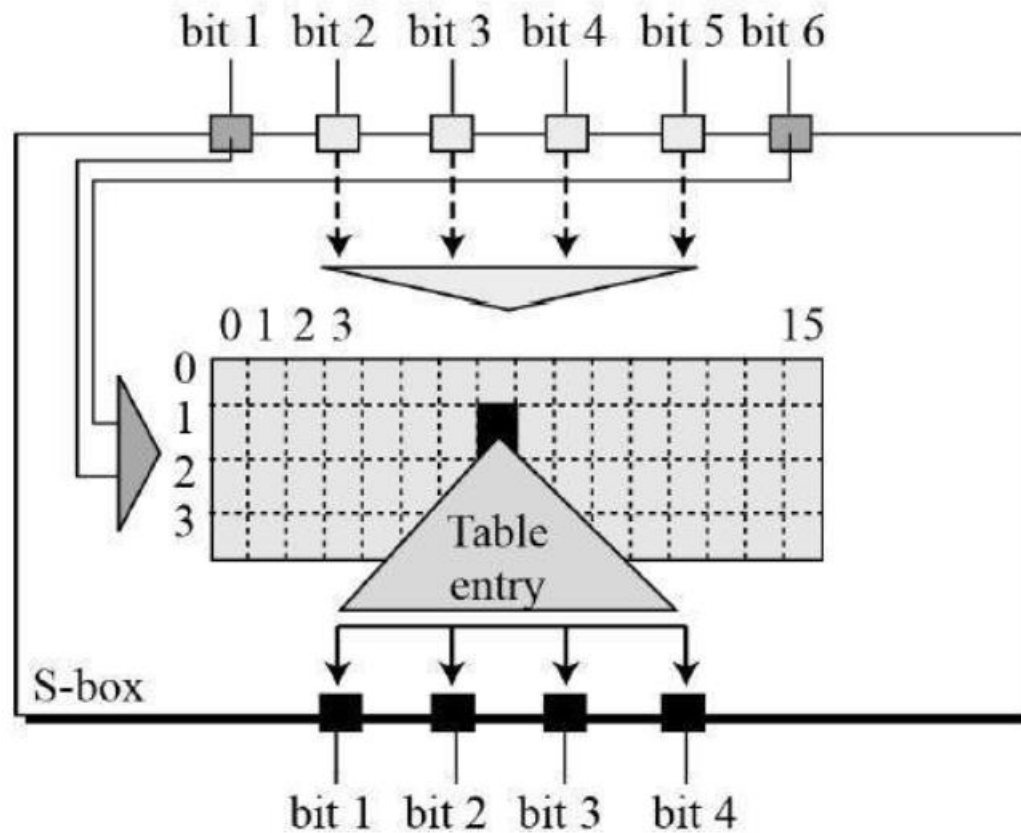
XOR

- ❑ After the expansion permutation, DES does XOR operation on the expanded right section and the round key.
- ❑ The round key is used only in this operation.

DES S-box

- ❑ The S-boxes carry out the real mixing (confusion).
- ❑ 8 "substitution boxes" or S-boxes
- ❑ Each S-box maps 6 bits to 4 bits
- ❑ There are a total of eight S-box tables.
The output of all eight s-boxes is then combined in to 32 bit section.

DES S-box



DES S-box

❑ Here is S-box number 1

input bits (0,5)



input bits (1,2,3,4)

		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111

00		1110	0100	1101	0001	0010	1111	1011	1000	0011	1010	0110	1100	0101	1001	0000	0111
01		0000	1111	0111	0100	1110	0010	1101	0001	1010	0110	1100	1011	1001	0101	0011	1000
10		0100	0001	1110	1000	1101	0110	0010	1011	1111	1100	1001	0111	0011	1010	0101	0000
11		1111	1100	1000	0010	0100	1001	0001	0111	0101	1011	0011	1110	1010	0000	0110	1101

DES S-box - Example

Row #	S_1	1	2	3	...	7									15	Column #
0	14	4	13	1	2	15	11	8	3	10	6	12	5	9	0	7
1	0	15	7	4	14	2	13	1	10	6	12	11	9	5	3	8
2	4	1	14	8	13	6	2	11	15	12	9	7	3	10	5	0
3	15	12	8	2	4	9	1	7	5	11	3	14	10	0	6	13

$S(i, j) < 16$, can be represented with 4 bits

Example: $B = 101111$

$b_1b_6 = 11 = \text{row } 3$

$b_2b_3b_4b_5 = 0111 = \text{column } 7$



$C = 7 = \overset{\text{hand}}{\underline{0111}}$

Another example: $B = 011011$, $C = ?$

Straight Permutation

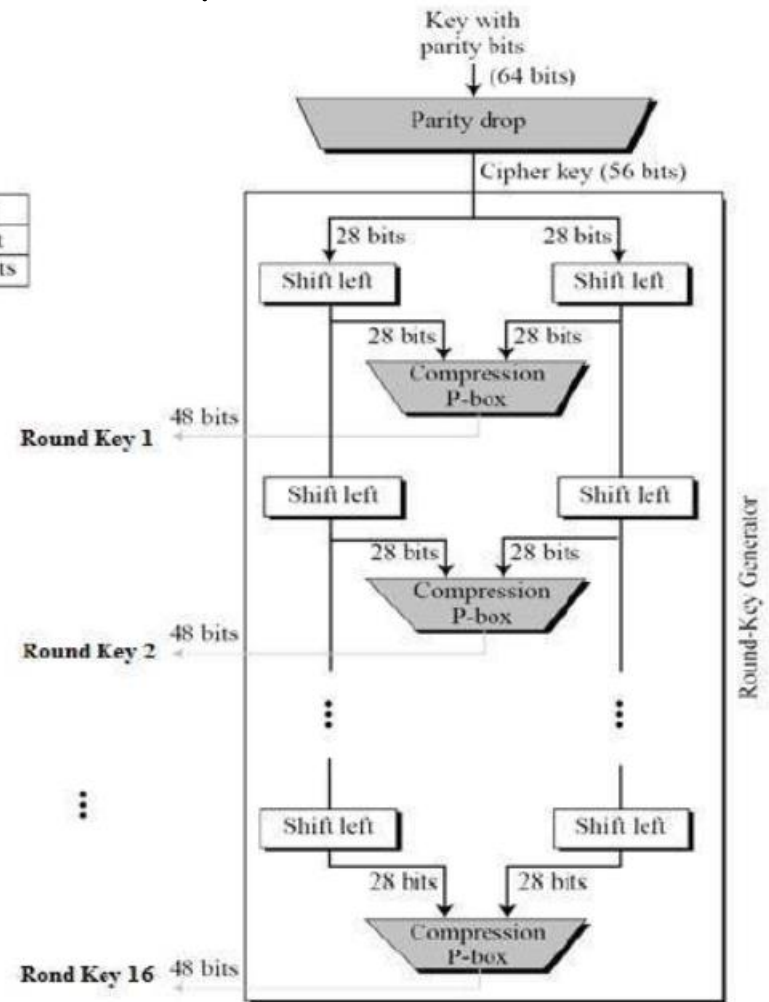
- The 32 bit output of S-boxes is then subjected to the straight permutation with rule shown below.

16	07	20	21	29	12	28	17
01	15	23	26	05	18	31	10
02	08	24	14	32	27	03	09
19	13	30	06	22	11	04	25

DES Subkey

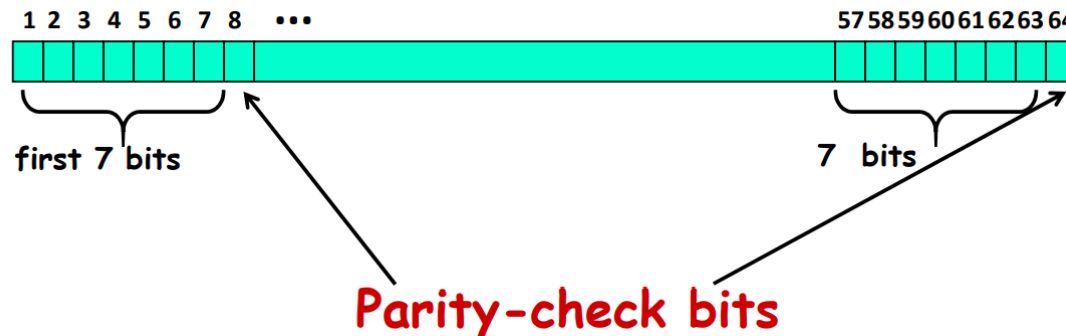
- ❑ The round-key generator creates sixteen 48-bit keys out of a 56-bit cipher key
- ❑ The logic for Parity drop, shifting, and Compression P-box is given in the DES description.

Shifting	
Rounds	Shift
1, 2, 9, 16	one bit
Others	two bits



Key length in DES

- ❑ In the DES specification, the key length is 64 bit: • 8 bytes; in each byte, the 8th bit is a parity-check bit
- ❑ Each parity-check bit is the XOR of the previous 7 bits



- ❑ Initial 64-bit key is transformed into a 56-bit key by discarding every 8th bit of the initial key

DES Subkey

- ❑ **Compression permutation** yields 48 bit subkey K_i from 56 bits of LK and RK
 - Since the key transformation process involves permutation as well as a selection of a 48-bit subset of the original 56-bit key it is called Compression Permutation.
- ❑ Bits 9,18,22,25 of LK omitted each round
- ❑ Bits 7,10,15,26 of RK omitted each round
 - i.e., bits (9, 18,22, 25, 35, 38, 43, 54) are omitted from the 56-bit key

14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

DES Analysis

- ❑ The DES satisfies both the desired properties of block cipher. These two properties make cipher very strong.
- ❑ **Avalanche effect** – A small change in plaintext results in the very great change in the ciphertext.
- ❑ **Completeness** – Each bit of ciphertext depends on many bits of plaintext.

Security of DES

- ❑ Security depends heavily on S-boxes
 - Everything else in DES is linear
- ❑ During the last few years, cryptanalysis have found some weaknesses in DES when key selected are weak keys. These keys shall be avoided at key generation
 - Weak keys: keys make the same sub-key to be generated in more than one round.
- ❑ DES has proved to be a very well designed block cipher. There have been no significant cryptanalytic attacks on DES other than exhaustive key search.

Security of DES

- ❑ DES has 4 weak keys

0000000 0000000
0000000 FFFFFFFF
FFFFFFFF 0000000
FFFFFFFF FFFFFFFF

- ❑ Using weak keys, the outcome of the Permuted Choice 1 (PC1) in the DES key schedule leads to round keys (K_1 --- K_{16}) being either all zeros, all ones or alternating zero-one patterns.
- ❑ Since all the subkeys are identical, and DES is a Feistel network, the encryption function becomes self-inverting; that is, encrypting twice with a weak key K produces the original plaintext.
 - $E_K(E_K(x))=x$ for all x , i.e., the encryption and the decryption are the same
- ❑ Weak keys should be avoided at key generation.

Applications of DES

- ❑ It is used in random number generation
- ❑ It is deployed when not-so-strong encryption is needed
- ❑ It is used to develop a new form of DES, called Triple DES (using a 168-bit key formed using three keys)

Decryption

- The steps involved in the steps for data decryption are:
 1. The order of the 16 48-bit keys is reversed such that key 16 becomes key 1, and so on.
 2. The steps for encryption are applied to the ciphertext.

Block Cipher Notation

- ❑ P = plaintext block
- ❑ C = ciphertext block
- ❑ Encrypt P with key K to get ciphertext C
 - $C = E(P, K)$
- ❑ Decrypt C with key K to get plaintext P
 - $P = D(C, K)$
- ❑ **Note:** $P = D(E(P, K), K)$ and $C = E(D(C, K), K)$
 - But $P \neq D(E(P, K_1), K_2)$ and $C \neq E(D(C, K_1), K_2)$ when $K_1 \neq K_2$

Block Cipher Modes

Multiple Blocks

- ❑ How to encrypt multiple blocks?
- ❑ Do we need a new key for each block?
 - If so, as impractical as a one-time pad!
- ❑ Encrypt each block independently?

Modes of Operation

- ❑ Many modes — we discuss 3 most popular
- ❑ Electronic Codebook (**ECB**) mode
 - Encrypt each block independently
 - Most obvious approach, but a bad idea
- ❑ Cipher Block Chaining (**CBC**) mode
 - Chain the blocks together
 - More secure than ECB, virtually no extra work
- ❑ Counter Mode (**CTR**) mode

ECB Mode

- ❑ Notation: $C = E(P, K)$
- ❑ Given plaintext $P_0, P_1, \dots, P_m, \dots$
- ❑ Most obvious way to use a block cipher:

Encrypt

$$C_0 = E(P_0, K)$$

$$C_1 = E(P_1, K)$$

$$C_2 = E(P_2, K) \dots$$

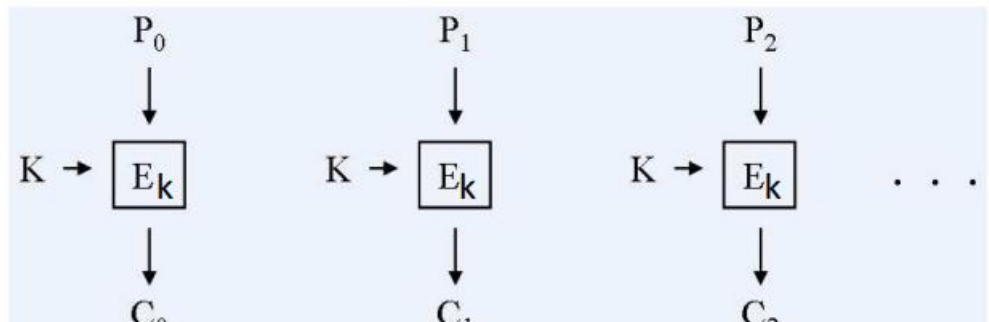
Decrypt

$$P_0 = D(C_0, K)$$

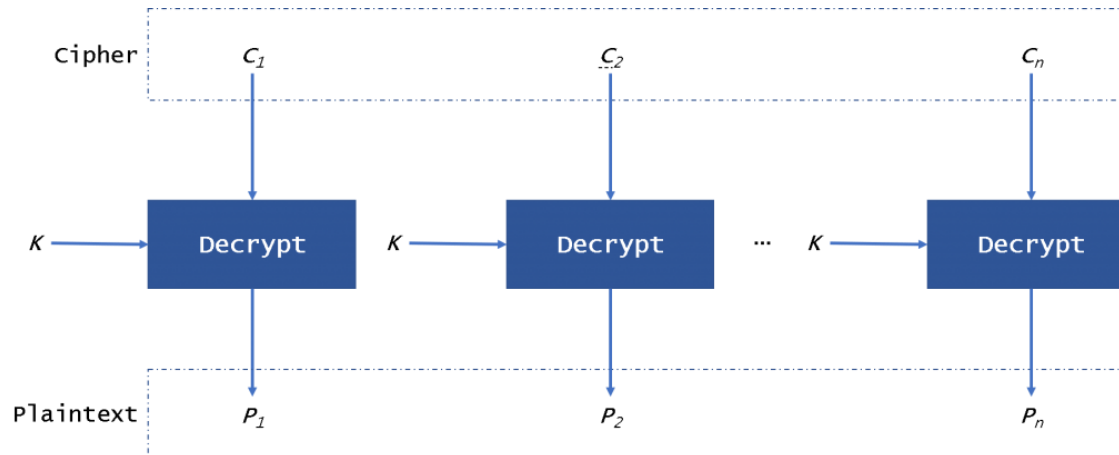
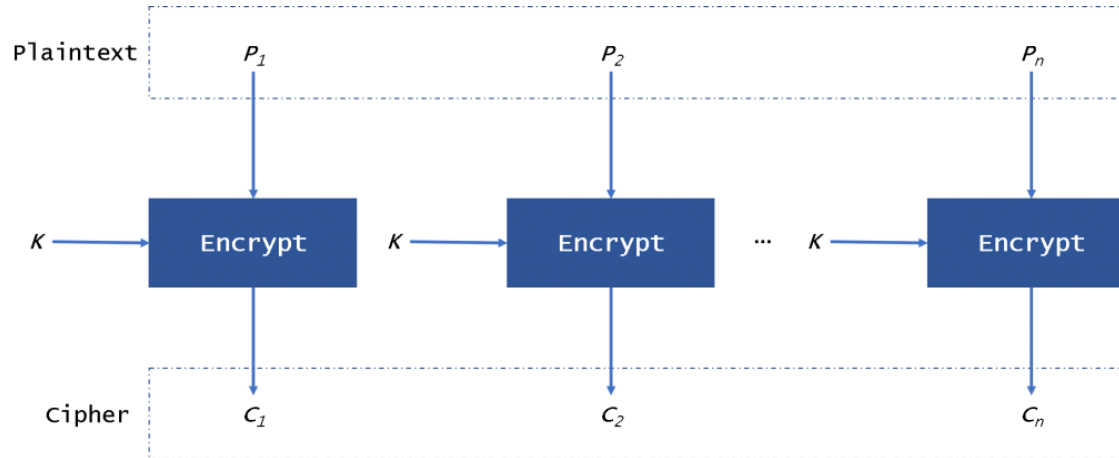
$$P_1 = D(C_1, K)$$

$$P_2 = D(C_2, K) \dots$$

- ❑ For fixed key K , this is “electronic” version of a codebook cipher (without additive)
 - With a different codebook for each key



ECB Mode



ECB Cut and Paste

- Suppose plaintext is

Alice digs Bob. Trudy digs Tom.

- Assuming 64-bit blocks and 8-bit ASCII:

P_0 = "Alice di", P_1 = "gs Bob. ",

P_2 = "Trudy di", P_3 = "gs Tom. "

- Ciphertext: C_0, C_1, C_2, C_3

- Trudy cuts and pastes: C_0, C_3, C_2, C_1

- Decrypts as

Alice digs Tom. Trudy digs Bob.

ECB Weakness

- ❑ Suppose $P_i = P_j$
- ❑ Then $C_i = C_j$ and Trudy knows $P_i = P_j$
- ❑ This gives Trudy some information, even if she does not know P_i or P_j
- ❑ Trudy might know P_i

ECB Properties

- ❑ Deterministic: the same data block gets encrypted the same way; this reveals patterns of data when a data block repeats.
- ❑ Malleable: reordering ciphertext results in reordered plaintext.
- ❑ Errors in one ciphertext block do not propagate.

CBC Mode

- ❑ Blocks are “chained” together
- ❑ A random initialization vector, or IV, is required to initialize CBC mode
- ❑ IV is random, but not secret

Encryption

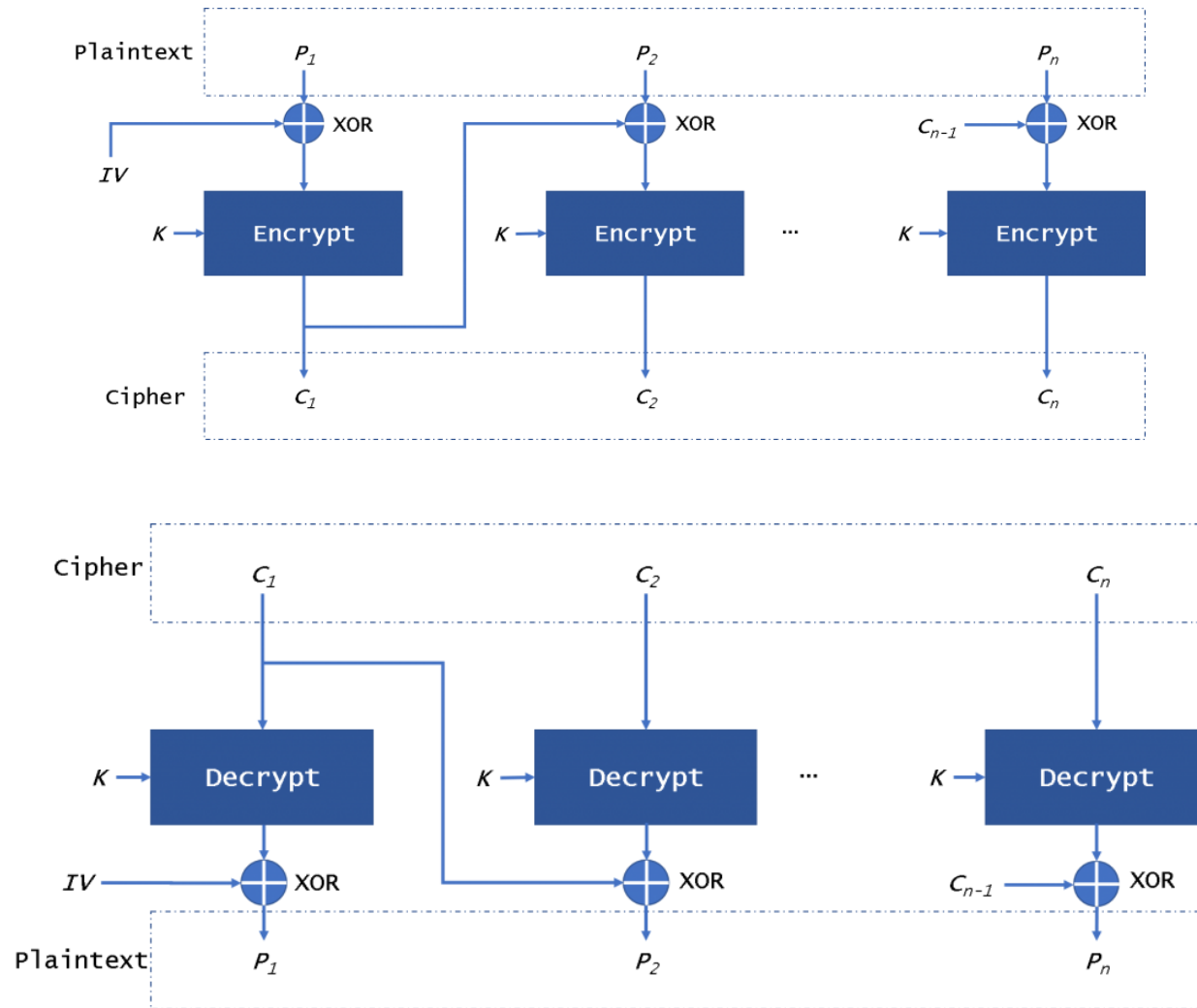
$$\begin{aligned}C_0 &= E(\text{IV} \oplus P_0, K), \\C_1 &= E(C_0 \oplus P_1, K), \\C_2 &= E(C_1 \oplus P_2, K), \dots\end{aligned}$$

Decryption

$$\begin{aligned}P_0 &= \text{IV} \oplus D(C_0, K), \\P_1 &= C_0 \oplus D(C_1, K), \\P_2 &= C_1 \oplus D(C_2, K), \dots\end{aligned}$$

- ❑ Analogous to classic codebook *with additive*

CBC Mode



CBC Mode

- ❑ Identical plaintext blocks yield different ciphertext blocks — this is very good!
- ❑ But what about errors in transmission?
 - If C_1 is garbled to, say, G then
$$P_1 \neq C_0 \oplus D(G, K), P_2 \neq G \oplus D(C_2, K)$$
 - But $P_3 = C_2 \oplus D(C_3, K), P_4 = C_3 \oplus D(C_4, K), \dots$
 - Automatically recovers from errors!
- ❑ Cut and paste is still possible, but more complex (and will cause garbles)

CBC Properties

- ❑ Randomized encryption: repeated text gets mapped to different encrypted data.
 - can be proven to be “secure” assuming that the block cipher has desirable properties and that random IV's are used
- ❑ A ciphertext block depends on all preceding plaintext blocks; reorder affects decryption
- ❑ Errors in one block propagate to two blocks
 - one bit error in C_j affects all bits in M_j and one bit in M_{j+1}
- ❑ Sequential encryption, cannot use parallel hardware
- ❑ Usage: chooses random IV and protects the integrity of IV

Counter Mode (CTR)

- CTR is popular for random access
- Use block cipher like a stream cipher

Encryption

$$C_0 = P_0 \oplus E(\text{IV}, K),$$

$$C_1 = P_1 \oplus E(\text{IV}+1, K),$$

$$C_2 = P_2 \oplus E(\text{IV}+2, K), \dots$$

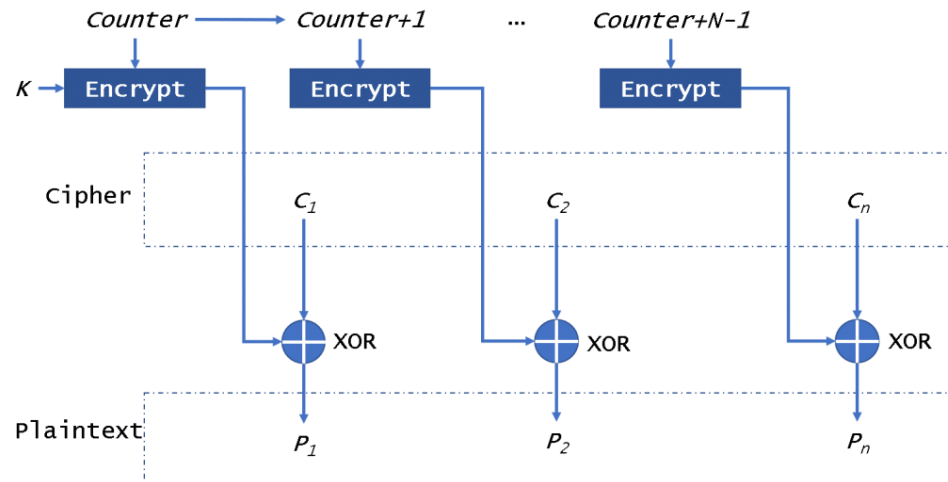
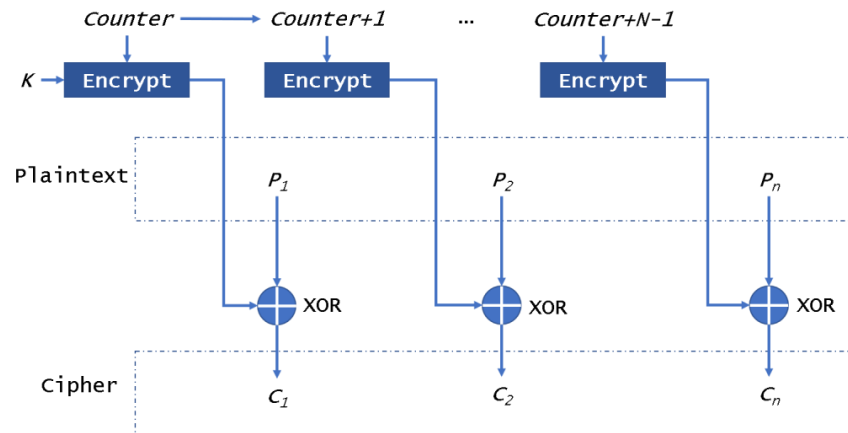
Decryption

$$P_0 = C_0 \oplus E(\text{IV}, K),$$

$$P_1 = C_1 \oplus E(\text{IV}+1, K),$$

$$P_2 = C_2 \oplus E(\text{IV}+2, K), \dots$$

Counter Mode (CTR)



CTR Properties

- ❑ Software and hardware efficiency: different blocks can be encrypted in parallel.
- ❑ Preprocessing: the encryption part can be done offline and when the message is known, just do the XOR.
- ❑ Random access: decryption of a block can be done in random order, very useful for hard-disk encryption.
- ❑ Messages of arbitrary length: ciphertext is the same length with the plaintext (i.e., no IV). 3

Integrity

Data Integrity

- ❑ **Integrity** — detect unauthorized writing (i.e., detect unauthorized mod of data)
- ❑ Example: Inter-bank fund transfers
 - Confidentiality may be nice, integrity is *critical*
- ❑ Encryption provides **confidentiality** (prevents unauthorized disclosure)
- ❑ Encryption alone does **not** provide integrity
 - One-time pad, ECB cut-and-paste, etc., etc.

MAC

- ❑ Message Authentication Code (MAC)
 - Used for data **integrity**
 - Integrity **not** the same as confidentiality
- ❑ MAC is computed as **CBC residue**
 - That is, compute CBC encryption, saving only final ciphertext block, the MAC
 - The MAC serves as a cryptographic checksum for data

MAC Computation

- MAC computation (assuming N blocks)

$$C_0 = E(IV \oplus P_0, K),$$

$$C_1 = E(C_0 \oplus P_1, K),$$

$$C_2 = E(C_1 \oplus P_2, K), \dots$$

$$C_{N-1} = E(C_{N-2} \oplus P_{N-1}, K) = \text{MAC}$$

- Send IV, $P_0, P_1, \dots, P_{N-1}$ and MAC
- Receiver does same computation and verifies that result agrees with MAC
- Both sender and receiver must know K

Does a MAC work?

- Suppose Alice has 4 plaintext blocks
- Alice computes
$$C_0 = E(IV \oplus P_0, K), C_1 = E(C_0 \oplus P_1, K),$$
$$C_2 = E(C_1 \oplus P_2, K), C_3 = E(C_2 \oplus P_3, K) = \text{MAC}$$
- Alice sends IV, P_0, P_1, P_2, P_3 and **MAC** to Bob
- Suppose Trudy changes P_1 to X
- Bob computes
$$C_0 = E(IV \oplus P_0, K), C_1 = E(C_0 \oplus X, K),$$
$$C_2 = E(C_1 \oplus P_2, K), C_3 = E(C_2 \oplus P_3, K) = \text{MAC} \neq \text{MAC}$$
- It works since error propagates into **MAC**
- Trudy can't make **MAC** == **MAC** without K

Confidentiality and Integrity

- ❑ Encrypt with one key, MAC with another key
- ❑ Why not use the same key?
 - Send last encrypted block (MAC) twice?
 - This cannot add any security!
- ❑ Using different keys to encrypt and compute MAC works, even if keys are related
 - But, twice as much work as encryption alone
 - Can do a little better — about 1.5 “encryptions”

Uses for Symmetric Crypto

❑ Confidentiality

- Transmitting data over insecure channel
- Secure storage on insecure media

❑ Integrity (MAC)